



[Latest](#) [Courses »](#) [Advice](#) [Bringers](#) [Everything Else »](#)

Android, MySQL, PHP, & JSON 5 | Login & Reg Android App Tutorial

[Home](#)

[Tutorial Series](#)

Android, MySQL, PHP, & JSON 5 | Login & Register Android App Tutorial

Posted By trav on Jun 10, 2013 | 0 comments

This entry is part 21 of 22 in the series [Android-Intermediate](#)



Creating the Android Application to Interact with our Remote Database!

In this Android Remote Databases mini series we'll create our Android application to parse the JSON data that our PHP scripts output. Our Android application will be able post data to our PHP scripts to register a new user, sign in, add a comment, or display all of the current comments.

Follow the Remote Databases for Android Series!

- [Part 1: Android Remote Databases and Servers Overview](#)
- [Part 2: Local Xampp Server and MySQL Database](#)
- [Part 3: Working with a Remote Server and MySQL](#)
- [Part 4: Creating Login and Registration PHP Scripts](#)
- **Part 5: Developing the Android Application**
- [Part 6: JSON Parsing and Android ListView Design](#)



Remote

Databases Part 5 Video Tutorial!

Coming soon..

Setting up our Android Project

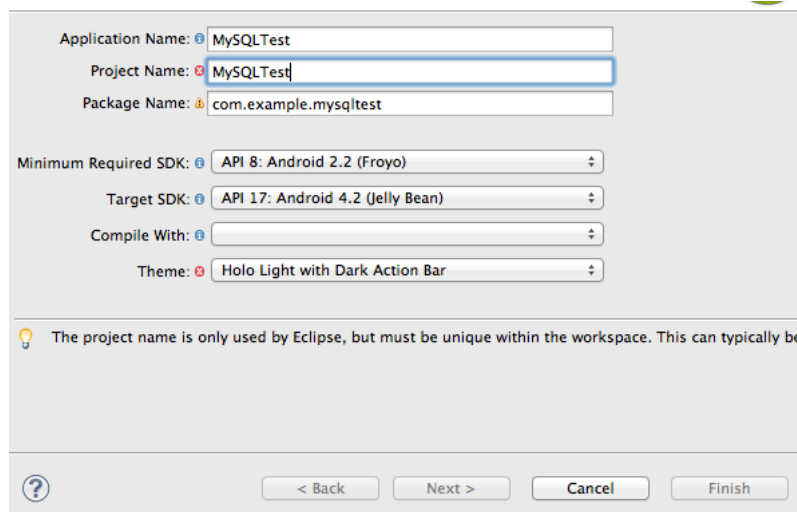
The time has finally come! We have everything setup for us to be able to interact with our MySQL remote database from within an Android application. So, let's create it! In this tutorial we will develop the login and register features of our app.

**You should be familiar with the basics of setting up an Android projects and the structure of an Android project. If at any time you are confused, you can check out our [Android Basics Course](#).

Step 1: Creating a New Android Project

Even though Android Studios just came out, I'm going to use Eclipse to setup my new Android Project. I gave it the following credentials:

- Application Name: MySQLTest
- Project Name: MySQLTest
- Package Name: com.example.mysqltest



Step 2: Setting Up Classes and Layouts

Create the following Java classes and xml layouts:

Java files

- Login.java
- Register.java
- AddComment.java
- ReadComments.java
- JSONParser.java

Android XML Layouts

- login.xml
- register.xml
- add_comment.xml
- read_comments.xml
- single_comment.xml

Step 3: Set up the Permissions and the Manifest

In order for use to interact with our PHP scripts and our remote MySQL database, we will need to make sure our Android application has the internet permission. Since we are modifying our manifest, we might as well add all of the activities we are going to have in our application as well.

AndroidManifest.xml

```

1 <?xml version="1.0" encoding="utf-8"?>
2 <manifest xmlns:android="http://schemas.android.com/apk/res/android"
3     package="com.example.mysqltest"
4     android:versionCode="1"
5     android:versionName="1.0" >
6
7     <uses-sdk
8         android:minSdkVersion="8"
9         android:targetSdkVersion="17" />
10    <uses-permission android:name="android.permission.INTERNET"/>
11
12    <application
13        android:allowBackup="true"
14        android:icon="@drawable/ic_launcher"
15        android:label="@string/app_name"
16        android:theme="@style/AppTheme" >
17        <activity
18            android:name="com.example.mysqltest.Login"
19            android:label="@string/app_name" >
20            <intent-filter>
21                <action android:name="android.intent.action.MAIN" />
22                <category android:name="android.intent.category.LAUNCHER" />
23            </intent-filter>
24        </activity>
25        <activity
26            android:name="com.example.mysqltest.Register"
27            android:label="@string/app_name" >
28        </activity>
29        <activity
30            android:name="com.example.mysqltest.AddComment"
31            android:label="@string/app_name" >
32        </activity>
33        <activity
34            android:name="com.example.mysqltest.ReadComments"
35            android:label="@string/app_name" >
36        </activity>
37    </application>
38 </manifest>

```

Step 4: Creating Simple Login and Register Layouts

I'm not the best when it comes to designing xml layouts. Especially, when I'm creating tutorials, so please bare with me here, these are going to be some ugly layouts for now.



Here is the image I'm using. Right-click on this bad boy, choose "Save as..", and save the image within your Android project's drawable folder as "arrowstars.png". (I have no idea why I have our logo labeled as arrowstars, but meh, it works!)

login.xml

```

1 <RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
2     xmlns:tools="http://schemas.android.com/tools"
3     android:layout_width="match_parent"
4     android:layout_height="match_parent"
5 >
6
7     <Button
8         android:id="@+id/register"
9         android:layout_width="wrap_content"
10        android:layout_height="wrap_content"
11        android:layout_alignLeft="@+id/login"
12        android:layout_alignParentBottom="true"
13        android:layout_alignRight="@+id/login"
14        android:layout_marginBottom="25dp"
15        android:text="Register" />
16

```

```

17 <Button
18     android:id="@+id/login"
19     android:layout_width="wrap_content"
20     android:layout_height="wrap_content"
21
22     android:layout_above="@+id/register"
23     android:layout_alignLeft="@+id/password"
24     android:layout_alignRight="@+id/password"
25     android:text="Login" />
26
27 <EditText
28     android:id="@+id/password"
29     android:layout_width="wrap_content"
30     android:layout_height="wrap_content"
31     android:layout_above="@+id/login"
32     android:layout_centerHorizontal="true"
33     android:ems="10"
34     android:inputType="textPassword" >
35
36     <requestFocus />
37 </EditText>
38
39 <TextView
40     android:id="@+id/textView2"
41     android:layout_width="wrap_content"
42     android:layout_height="wrap_content"
43     android:layout_alignParentTop="true"
44     android:layout_centerHorizontal="true"
45     android:layout_marginTop="17dp"
46     android:gravity="center"
47     android:text="Android Remote Server Tutorial"
48     android:textAppearance="?android:attr/textAppearanceLarge"
49     android:textStyle="bold" />
50
51 <ImageView
52     android:id="@+id/imageView1"
53     android:layout_width="wrap_content"
54     android:layout_height="wrap_content"
55     android:layout_below="@+id/textView2"
56     android:layout_centerHorizontal="true"
57     android:src="@drawable/arrowstars" />
58
59 <TextView
60     android:id="@+id/TextView01"
61     android:layout_width="wrap_content"
62     android:layout_height="wrap_content"
63     android:layout_above="@+id/password"
64     android:layout_alignLeft="@+id/password"
65     android:layout_marginLeft="22dp"
66     android:text="Password" />
67
68 <EditText
69     android:id="@+id/username"
70     android:layout_width="wrap_content"
71     android:layout_height="wrap_content"
72     android:layout_above="@+id/TextView01"
73     android:layout_centerHorizontal="true"
74     android:ems="10" />
75
76 <TextView
77     android:id="@+id/textView1"
78     android:layout_width="wrap_content"
79     android:layout_height="wrap_content"
80     android:layout_alignRight="@+id/TextView01"
81     android:layout_centerVertical="true"
82     android:text="Username" />
83 </RelativeLayout>

```

register.xml

```

1 <RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
2     xmlns:tools="http://schemas.android.com/tools"
3     android:layout_width="match_parent"
4     android:layout_height="match_parent"
5 >
6
7 <ImageView
8     android:id="@+id/imageView1"
9     android:layout_width="wrap_content"
10    android:layout_height="wrap_content"
11    android:layout_below="@+id/textView2"
12    android:layout_centerHorizontal="true"
13    android:src="@drawable/arrowstars" />
14
15 <TextView
16     android:id="@+id/textView1"
17     android:layout_width="wrap_content"
18     android:layout_height="wrap_content"
19     android:layout_alignLeft="@+id/password"
20     android:layout_centerVertical="true"

```

```

21         android:text="Username" />
22
23     <EditText
24         android:id="@+id/username"
25         android:layout_width="wrap_content"
26         android:layout_height="wrap_content"
27         android:layout_alignLeft="@+id/textView1"
28         android:layout_below="@+id/textView1"
29         android:ems="10" />
30
31     <TextView
32         android:id="@+id/TextView01"
33         android:layout_width="wrap_content"
34         android:layout_height="wrap_content"
35         android:layout_alignLeft="@+id/username"
36         android:layout_below="@+id/username"
37         android:text="Password" />
38
39     <TextView
40         android:id="@+id/textView2"
41         android:layout_width="wrap_content"
42         android:layout_height="wrap_content"
43         android:layout_alignParentLeft="true"
44         android:layout_alignParentTop="true"
45         android:layout_marginTop="16dp"
46         android:gravity="center"
47         android:text="Android Remote Server Tutorial"
48         android:textAppearance="?android:attr/textAppearanceLarge"
49         android:textStyle="bold" />
50
51     <EditText
52         android:id="@+id/password"
53         android:layout_width="wrap_content"
54         android:layout_height="wrap_content"
55         android:layout_below="@+id/TextView01"
56         android:layout_centerHorizontal="true"
57         android:ems="10"
58         android:inputType="textPassword" />
59
60     <Button
61         android:id="@+id/register"
62         android:layout_width="wrap_content"
63         android:layout_height="wrap_content"
64         android:layout_alignRight="@+id/password"
65         android:layout_below="@+id/password"
66         android:text="Register" />
67
68 </RelativeLayout>

```

I know I'm not using the standard string references for my text values, but don't worry about that, we can fix everything later.



Step 5:
Creating
Our
Login

and Register Activities.

Now, the fun begins, we need to test our login system. First, let create a simple success page for our ReadComments.java and read_comments.xml

read_comments.xml

```

1 <?xml version="1.0" encoding="utf-8"?>
2 <LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
3     android:layout_width="match_parent"
4     android:layout_height="match_parent"
5     android:background="#fff" >
6
7     <TextView
8         android:id="@+id/textView1"
9         android:layout_width="wrap_content"
10        android:layout_height="wrap_content"
11        android:text="Success!"
12        android:textAppearance="?android:attr/textAppearanceLarge" />
13
14 </LinearLayout>

```

ReadComments.java

```

1 package com.example.mysqltest;
2
3 import android.app.Activity;
4 import android.os.Bundle;
5
6 public class ReadComments extends Activity{

```

```

7
8     @Override
9     protected void onCreate(Bundle savedInstanceState) {
10         // TODO Auto-generated method stub
11         super.onCreate(savedInstanceState);
12         setContentView(R.layout.read_comments);
13     }
14 }

```

That was simple enough! Now, we need to create a class that will help us parse some JSON information. There are a lot of [JSON parsing classes](#) for Android on the internet you can use, or you could create your own. To save time, I'm going to use one I found on the internet, and modify it to our needs.

```

1 package com.example.mysqltest;
2
3 import java.io.BufferedReader;
4 import java.io.IOException;
5 import java.io.InputStream;
6 import java.io.InputStreamReader;
7 import java.io.UnsupportedEncodingException;
8
9 import org.apache.http.HttpEntity;
10 import org.apache.http.HttpResponse;
11 import org.apache.http.client.ClientProtocolException;
12 import org.apache.http.client.methods.HttpPost;
13 import org.apache.http.impl.client.DefaultHttpClient;
14 import org.json.JSONException;
15 import org.json.JSONObject;
16
17 import android.util.Log;
18
19 public class JSONParser {
20
21     static InputStream is = null;
22     static JSONObject jsonObj = null;
23     static String json = "";
24
25     // constructor
26     public JSONParser() {
27
28     }
29
30     public JSONObject getJSONFromUrl(String url) {
31
32         // Making HTTP request
33         try {
34             // defaultHttpClient
35             DefaultHttpClient httpClient = new DefaultHttpClient();
36             HttpPost httpPost = new HttpPost(url);
37
38             HttpResponse httpResponse = httpClient.execute(httpPost);
39             HttpEntity httpEntity = httpResponse.getEntity();
40             is = httpEntity.getContent();
41
42         } catch (UnsupportedEncodingException e) {
43             e.printStackTrace();
44         } catch (ClientProtocolException e) {
45             e.printStackTrace();
46         } catch (IOException e) {
47             e.printStackTrace();
48         }
49
50         try {
51             BufferedReader reader = new BufferedReader(new InputStreamReader(
52                 is, "iso-8859-1"), 8);
53             StringBuilder sb = new StringBuilder();
54             String line = null;
55             while ((line = reader.readLine()) != null) {
56                 sb.append(line + "\n");
57             }
58             is.close();
59             json = sb.toString();
60         } catch (Exception e) {
61             Log.e("Buffer Error", "Error converting result " + e.toString());
62         }
63
64         // try parse the string to a JSON object
65         try {
66             jsonObj = new JSONObject(json);
67         } catch (JSONException e) {
68             Log.e("JSON Parser", "Error parsing data " + e.toString());
69         }
70
71         // return JSON String
72         return jsonObj;
73     }
74 }
75 }

```

This JSONParser.java class may look like a bunch of baloney to you right now, but this is the class that allows us to interpret the JSON data our php web service spits out. With this class we will be able to read whether or not a login was successful (with either a 0 or 1). We will also be able to list out all of our posts in an orderly fashion using this JSON parser. We will go over the details later. As for now lets setup our Login Activity:

Login.java v0.1 – Basics

```

1 package com.example.mysqltest;
2
3 import android.app.Activity;
4 import android.app.AlertDialog;
5 import android.content.Intent;
6 import android.os.AsyncTask;
7 import android.os.Bundle;
8 import android.view.View;
9
10 import android.view.View.OnClickListener;
11 import android.widget.Button;
12 import android.widget.EditText;
13
14 public class Login extends Activity implements OnClickListener{
15
16     private EditText user, pass;
17     private Button mSubmit, mRegister;
18
19     // Progress Dialog
20     private ProgressDialog pDialog;
21
22     // JSON parser class
23     JSONParser jsonParser = new JSONParser();
24
25     //php login script location:
26
27     //localhost :
28     //testing on your device
29     //put your local ip instead, on windows, run CMD > ipconfig
30     //or in mac's terminal type ifconfig and look for the ip under en0 or en1
31     // private static final String LOGIN_URL =
32     "http://xxx.xxx.x.x:1234/webservice/login.php";
33
34     //testing on Emulator:
35     private static final String LOGIN_URL =
36     "http://10.0.2.2:1234/webservice/login.php";
37
38     //testing from a real server:
39     //private static final String LOGIN_URL =
40     "http://www.yourdomain.com/webservice/login.php";
41
42     //JSON element ids from repsonse of php script:
43     private static final String TAG_SUCCESS = "success";
44     private static final String TAG_MESSAGE = "message";
45
46     @Override
47     protected void onCreate(Bundle savedInstanceState) {
48         // TODO Auto-generated method stub
49         super.onCreate(savedInstanceState);
50         setContentView(R.layout.login);
51
52         //setup input fields
53         user = (EditText)findViewById(R.id.username);
54         pass = (EditText)findViewById(R.id.password);
55
56         //setup buttons
57         mSubmit = (Button)findViewById(R.id.login);
58         mRegister = (Button)findViewById(R.id.register);
59
60         //register listeners
61         mSubmit.setOnClickListener(this);
62         mRegister.setOnClickListener(this);
63
64     }
65
66     @Override
67     public void onClick(View v) {
68         // determine which button was pressed:
69         switch (v.getId()) {
70             case R.id.login:
71                 new AttemptLogin().execute();
72                 break;
73             case R.id.register:
74                 Intent i = new Intent(this, Register.class);
75                 startActivity(i);
76                 break;
77             default:
78                 break;
79         }
80     }
81

```

```

79 //AsyncTask is a separete thread than the thread that runs the GUI
80 //Any type of networking should be done with asyncnctask.
81 class AttemptLogin extends AsyncTask<String, String, String> {
82
83     //three methods get called, first preExecture, then do in background, and
once do
84     //in back ground is completed, the onPostExecute method will be called.
85
86     @Override
87     protected void onPreExecute() {
88         super.onPreExecute();
89     }
90
91
92     @Override
93     protected String doInBackground(String... args) {
94
95         return null;
96     }
97
98
99     protected void onPostExecute(String file_url) {
100
101     }
102
103 }
104
105 }

```

This java file looks pretty basic so far. The important part is that we are using AsyncTask because connecting our php scripts may take some time, and therefore we don't want to use the same thread that runs the GUI of our application. If we did, it would really slow things down, freeze usability for our user, and maybe even crash our application. AsyncTask will create another thread to execute the tasks we want completed.

You also want to note where your PHP scripts are located. If you are using Xampp as a local server, make sure you uncomment the appropriate LOGIN_URL. If your scripts are on a shared server somewhere, use your domain name as the LOGIN_URL.

Here is the break down of what we want our login java class to do:

- If the register button is clicked, open the register activity.
- If the login button is clicked, create a dialog that pops up before trying to connect to the scripts.
- Once the dialog is displayed, get the inputs from our "username" and "password" EditTexts.
- POST that data to our URL that handles logging in.
- Retrieve the JSON response from our server.
- Interpret the response.
- If the response element "success" is 1, finish the Login Activity, and open the ReadComments Activity.
- If the response element "success" is 0, do nothing.
- In either case, we will close our dialog.
- Lastly, we will display the JSON response for the element "message" as a toast.

Awesome, let's make it happen.

Login.java v0.1 – Basics

```

1 package com.example.mysqltest;
2
3 import java.util.ArrayList;
4 import java.util.List;
5
6 import org.apache.http.NameValuePair;
7 import org.apache.http.message.BasicNameValuePair;
8 import org.json.JSONException;
9 import org.json.JSONObject;
10
11 import android.app.Activity;
12 import android.app.AlertDialog;
13 import android.content.Intent;
14 import android.os.AsyncTask;
15 import android.os.Bundle;
16 import android.util.Log;
17 import android.view.View;
18 import android.view.View.OnClickListener;
19 import android.widget.Button;
20 import android.widget.EditText;
21 import android.widget.Toast;
22
23 public class Login extends Activity implements OnClickListener{
24
25     private EditText user, pass;
26     private Button mSubmit, mRegister;
27

```



```

28     // Progress Dialog
29     private ProgressDialog pDialog;
30
31     // JSON parser class
32     JSONParser jsonParser = new JSONParser();
33
34     //php login script location:
35
36     //localhost :
37     //testing on your device
38     //put your local ip instead, on windows, run CMD > ipconfig
39     //or in mac's terminal type ifconfig and look for the ip under en0 or en1
40     // private static final String LOGIN_URL =
41     "http://xxx.xxx.x.x:1234/webservice/login.php";
42
43     //testing on Emulator:
44     private static final String LOGIN_URL =
45     "http://10.0.2.2:1234/webservice/login.php";
46
47     //testing from a real server:
48     //private static final String LOGIN_URL =
49     "http://www.yourdomain.com/webservice/login.php";
50
51     //JSON element ids from repsonse of php script:
52     private static final String TAG_SUCCESS = "success";
53     private static final String TAG_MESSAGE = "message";
54
55     @Override
56     protected void onCreate(Bundle savedInstanceState) {
57         // TODO Auto-generated method stub
58         super.onCreate(savedInstanceState);
59         setContentView(R.layout.login);
60
61         //setup input fields
62         user = (EditText)findViewById(R.id.username);
63         pass = (EditText)findViewById(R.id.password);
64
65         //setup buttons
66         mSubmit = (Button)findViewById(R.id.login);
67         mRegister = (Button)findViewById(R.id.register);
68
69         //register listeners
70         mSubmit.setOnClickListener(this);
71         mRegister.setOnClickListener(this);
72     }
73
74     @Override
75     public void onClick(View v) {
76         // TODO Auto-generated method stub
77         switch (v.getId()) {
78             case R.id.login:
79                 new AttemptLogin().execute();
80                 break;
81             case R.id.register:
82                 Intent i = new Intent(this, Register.class);
83                 startActivity(i);
84                 break;
85             default:
86                 break;
87         }
88     }
89
90     class AttemptLogin extends AsyncTask<String, String, String> {
91
92         /**
93          * Before starting background thread Show Progress Dialog
94          */
95         boolean failure = false;
96
97         @Override
98         protected void onPreExecute() {
99             super.onPreExecute();
100             pDialog = new ProgressDialog(Login.this);
101             pDialog.setMessage("Attempting login...");
102             pDialog.setIndeterminate(false);
103             pDialog.setCancelable(true);
104             pDialog.show();
105         }
106
107         @Override
108         protected String doInBackground(String... args) {
109             // TODO Auto-generated method stub
110             // Check for success tag
111             int success;
112             String username = user.getText().toString();
113             String password = pass.getText().toString();
114             try {
115                 // Building Parameters

```

```

115 List<NameValuePair> params = new ArrayList<NameValuePair>();
116 params.add(new BasicNameValuePair("username", username));
117 params.add(new BasicNameValuePair("password", password));
118
119 Log.d("request!", "starting");
120 // getting product details by making HTTP request
121 JSONObject json = jsonParser.makeHttpRequest(
122     LOGIN_URL, "POST", params);
123
124 // check your log for json response
125 Log.d("Login attempt", json.toString());
126
127 // json success tag
128 success = json.getInt(TAG_SUCCESS);
129 if (success == 1) {
130     Log.d("Login Successful!", json.toString());
131     Intent i = new Intent(Login.this, ReadComments.class);
132     finish();
133     startActivity(i);
134     return json.getString(TAG_MESSAGE);
135 }else{
136     Log.d("Login Failure!", json.getString(TAG_MESSAGE));
137     return json.getString(TAG_MESSAGE);
138 }
139 } catch (JSONException e) {
140     e.printStackTrace();
141 }
142
143 return null;
144
145
146 }
147 /**
148  * After completing background task Dismiss the progress dialog
149  * **/
150 protected void onPostExecute(String file_url) {
151     // dismiss the dialog once product deleted
152     pDialog.dismiss();
153     if (file_url != null){
154         Toast.makeText(Login.this, file_url, Toast.LENGTH_LONG).show();
155     }
156 }
157 }
158
159 }
160
161 }

```

Reading JSON data is as simple as that! We simply POST the information to the url we want within the make HTTP request, and we parse the result! You can now look at the JSONParser.java class and see that it is quite a simple class. All that it does is determine what type of method to use, either POST or GET, and then it makes a httprequest and gets the response. Lastly, the JSONParser class will use a buffer reader to get the different elements of the JSON object.

You also may have noticed that we are calling a method from JSONParser class called makeHttpRequest. This is a method that will read the JSON data and put everything in the appropriate list. Let's add modify our JSONParser class to include this method.

JSONParser.java

```

1 package com.example.mysqltest;
2
3 import java.io.BufferedReader;
4 import java.io.IOException;
5 import java.io.InputStream;
6 import java.io.InputStreamReader;
7 import java.io.UnsupportedEncodingException;
8 import java.util.List;
9
10 import org.apache.http.HttpEntity;
11 import org.apache.http.HttpResponse;
12 import org.apache.http.NameValuePair;
13 import org.apache.http.client.ClientProtocolException;
14 import org.apache.http.client.entity.UrlEncodedFormEntity;
15 import org.apache.http.client.methods.HttpGet;
16 import org.apache.http.client.methods.HttpPost;
17 import org.apache.http.client.utils.URLEncodedUtils;
18 import org.apache.http.impl.client.DefaultHttpClient;
19 import org.json.JSONException;
20 import org.json.JSONObject;
21
22 import android.util.Log;
23
24 public class JSONParser {
25
26     static InputStream is = null;

```

```

27 static JSONObject jsonObj = null;
28 static String json = "";
29
30 // constructor
31 public JSONParser() {
32
33 }
34
35
36 public JSONObject getJSONFromUrl(final String url) {
37
38     // Making HTTP request
39     try {
40         // Construct the client and the HTTP request.
41         DefaultHttpClient httpClient = new DefaultHttpClient();
42         HttpPost httpPost = new HttpPost(url);
43
44         // Execute the POST request and store the response locally.
45         HttpResponse httpResponse = httpClient.execute(httpPost);
46         // Extract data from the response.
47         HttpEntity httpEntity = httpResponse.getEntity();
48         // Open an inputStream with the data content.
49         is = httpEntity.getContent();
50
51     } catch (UnsupportedEncodingException e) {
52         e.printStackTrace();
53     } catch (ClientProtocolException e) {
54         e.printStackTrace();
55     } catch (IOException e) {
56         e.printStackTrace();
57     }
58
59     try {
60         // Create a BufferedReader to parse through the inputStream.
61         BufferedReader reader = new BufferedReader(new InputStreamReader(
62             is, "iso-8859-1"), 8);
63         // Declare a string builder to help with the parsing.
64         StringBuilder sb = new StringBuilder();
65         // Declare a string to store the JSON object data in string form.
66         String line = null;
67
68         // Build the string until null.
69         while ((line = reader.readLine()) != null) {
70             sb.append(line + "\n");
71         }
72
73         // Close the input stream.
74         is.close();
75         // Convert the string builder data to an actual string.
76         json = sb.toString();
77     } catch (Exception e) {
78         Log.e("Buffer Error", "Error converting result " + e.toString());
79     }
80
81     // Try to parse the string to a JSON object
82     try {
83         jsonObj = new JSONObject(json);
84     } catch (JSONException e) {
85         Log.e("JSON Parser", "Error parsing data " + e.toString());
86     }
87
88     // Return the JSON Object.
89     return jsonObj;
90 }
91
92
93
94 // function get json from url
95 // by making HTTP POST or GET method
96 public JSONObject makeHttpRequest(String url, String method,
97     List<NameValuePair> params) {
98
99     // Making HTTP request
100     try {
101
102         // check for request method
103         if(method == "POST"){
104             // request method is POST
105             // defaultHttpClient
106             DefaultHttpClient httpClient = new DefaultHttpClient();
107             HttpPost httpPost = new HttpPost(url);
108             httpPost.setEntity(new UrlEncodedFormEntity(params));
109
110             HttpResponse httpResponse = httpClient.execute(httpPost);
111             HttpEntity httpEntity = httpResponse.getEntity();
112             is = httpEntity.getContent();
113
114         }else if(method == "GET"){
115             // request method is GET
116             DefaultHttpClient httpClient = new DefaultHttpClient();
117             String paramString = URLEncodedUtils.format(params, "utf-8");

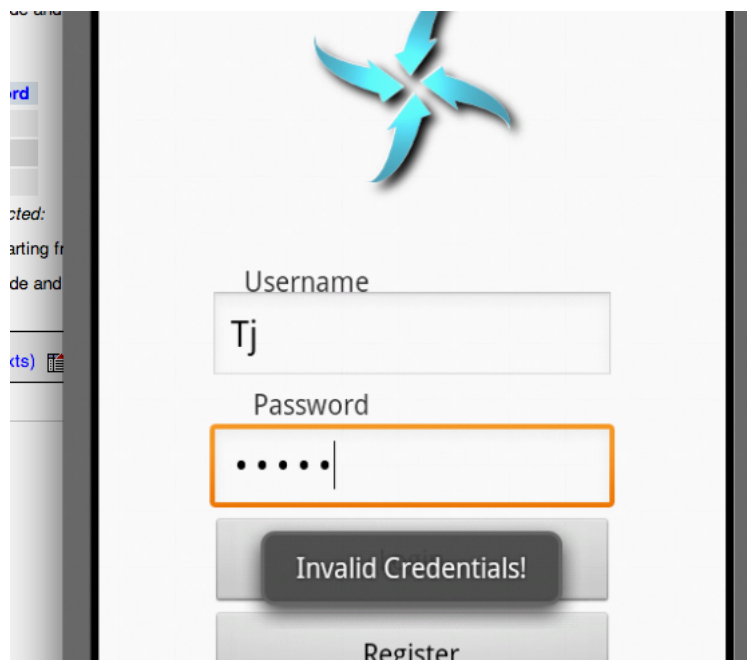
```

```

118         url += "?" + paramString;
119         HttpGet httpGet = new HttpGet(url);
120
121         HttpResponse httpResponse = httpClient.execute(httpGet);
122         HttpEntity httpEntity = httpResponse.getEntity();
123         is = httpEntity.getContent();
124     }
125
126     } catch (UnsupportedEncodingException e) {
127         e.printStackTrace();
128     } catch (ClientProtocolException e) {
129         e.printStackTrace();
130     } catch (IOException e) {
131         e.printStackTrace();
132     }
133
134     try {
135         BufferedReader reader = new BufferedReader(new InputStreamReader(
136             is, "iso-8859-1"), 8);
137         StringBuilder sb = new StringBuilder();
138         String line = null;
139
140         while ((line = reader.readLine()) != null) {
141             sb.append(line + "\n");
142         }
143         is.close();
144         json = sb.toString();
145     } catch (Exception e) {
146         Log.e("Buffer Error", "Error converting result " + e.toString());
147     }
148
149     // try parse the string to a JSON object
150     try {
151         jsonObj = new JSONObject(json);
152     } catch (JSONException e) {
153         Log.e("JSON Parser", "Error parsing data " + e.toString());
154     }
155
156     // return JSON String
157     return jsonObj;
158 }
159 }

```

Another thing I wanted to mention is that, we don't want to create a toast within the "doInBackground" method of our AsyncTask class, instead, we want to pass the response to the onPostExecute method and display the toast there! Again, this toast is displaying whatever the "message" JSON element contains, and therefore, as a front-end Android developer, you wouldn't have to worry about what it says since that would be the back-end programmer's job. As you can see here, this is what is displayed when you try to login with an unregistered username:



Now, let's outline what we want the **Register.java** class should do:

- Try to register user.
- If JSON element "success" is 1, display success message and close registration activity.
- If JSON element "success" is 0, display message explaining why registration is unsuccessful.
- That's it.

Register.java

```

1 package com.example.mysqltest;
2
3 import java.util.ArrayList;
4 import java.util.List;
5 import org.apache.http.NameValuePair;
6 import org.apache.http.message.BasicNameValuePair;
7 import org.json.JSONException;
8 import org.json.JSONObject;
9 import android.app.Activity;
10 import android.app.ProgressDialog;
11 import android.os.AsyncTask;
12 import android.os.Bundle;
13 import android.util.Log;
14 import android.view.View;
15 import android.view.View.OnClickListener;
16 import android.widget.Button;
17 import android.widget.EditText;
18 import android.widget.Toast;
19
20 public class Register extends Activity implements OnClickListener{
21
22     private EditText user, pass;
23     private Button mRegister;
24
25     // Progress Dialog
26     private ProgressDialog pDialog;
27
28     // JSON parser class
29     JSONParser jsonParser = new JSONParser();
30
31     //php login script
32
33     //localhost :
34     //testing on your device
35     //put your local ip instead, on windows, run CMD > ipconfig
36     //or in mac's terminal type ifconfig and look for the ip under en0 or en1
37     // private static final String LOGIN_URL =
38     "http://xxx.xxx.x.x:1234/webservice/register.php";
39
40     //testing on Emulator:
41     private static final String LOGIN_URL =
42     "http://10.0.2.2:1234/webservice/register.php";
43
44     //testing from a real server:
45     //private static final String LOGIN_URL =
46     "http://www.yourdomain.com/webservice/register.php";
47
48     //ids
49     private static final String TAG_SUCCESS = "success";
50     private static final String TAG_MESSAGE = "message";
51
52     @Override
53     protected void onCreate(Bundle savedInstanceState) {
54         // TODO Auto-generated method stub
55         super.onCreate(savedInstanceState);
56         setContentView(R.layout.register);
57
58         user = (EditText)findViewById(R.id.username);
59         pass = (EditText)findViewById(R.id.password);
60
61         mRegister = (Button)findViewById(R.id.register);
62         mRegister.setOnClickListener(this);
63
64     }
65
66     @Override
67     public void onClick(View v) {
68         // TODO Auto-generated method stub
69
70         new CreateUser().execute();
71
72     }
73
74     class CreateUser extends AsyncTask<String, String, String> {
75
76         /**
77          * Before starting background thread Show Progress Dialog
78          */
79         boolean failure = false;
80
81         @Override
82         protected void onPreExecute() {
83             super.onPreExecute();
84             pDialog = new ProgressDialog(Register.this);
85             pDialog.setMessage("Creating User...");
86             pDialog.setIndeterminate(false);
87             pDialog.setCancelable(true);
88             pDialog.show();

```

```

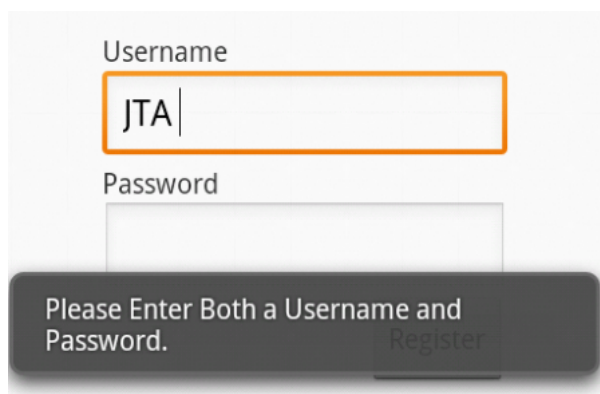
86     }
87
88     @Override
89     protected String doInBackground(String... args) {
90         // TODO Auto-generated method stub
91         // Check for success tag
92         int success;
93         String username = user.getText().toString();
94         String password = pass.getText().toString();
95         try {
96             // Building Parameters
97             List<NameValuePair> params = new ArrayList<NameValuePair>();
98             params.add(new BasicNameValuePair("username", username));
99             params.add(new BasicNameValuePair("password", password));
100
101             Log.d("request!", "starting");
102
103             //Posting user data to script
104             JSONObject json = jsonParser.makeHttpRequest(
105                 LOGIN_URL, "POST", params);
106
107             // full json response
108             Log.d("Login attempt", json.toString());
109
110             // json success element
111             success = json.getInt(TAG_SUCCESS);
112             if (success == 1) {
113                 Log.d("User Created!", json.toString());
114                 finish();
115                 return json.getString(TAG_MESSAGE);
116             } else {
117                 Log.d("Login Failure!", json.getString(TAG_MESSAGE));
118                 return json.getString(TAG_MESSAGE);
119             }
120         } catch (JSONException e) {
121             e.printStackTrace();
122         }
123
124         return null;
125     }
126
127     /**
128     * After completing background task Dismiss the progress dialog
129     * **/
130     protected void onPostExecute(String file_url) {
131         // dismiss the dialog once product deleted
132         pDialog.dismiss();
133         if (file_url != null) {
134             Toast.makeText(Register.this, file_url, Toast.LENGTH_LONG).show();
135         }
136     }
137
138 }
139
140 }
141
142 }

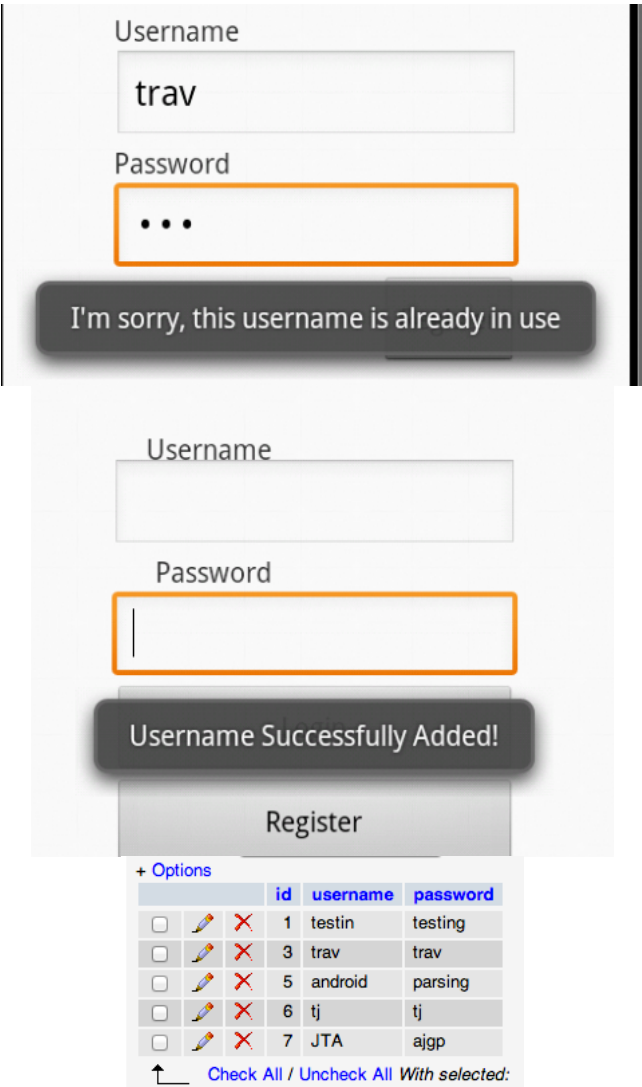
```

```

{"success":0,"message":"Please Enter Both a Username and Password."}

```





The registration activity looks pretty similar to the login activity. All we do, is try to register a user by POSTing data to our registration script, if it is successful, cool. If not, display a message explaining why.

Download the Source!

[Download Source](#) That's it for this tutorial! [Support the Cause!](#)

We have successfully communicated with our remote server and MySQL database! In the next tutorial we will create our other two activities that will post comments and parse comments. We will also work a little bit with designing a listview to look a bit more stylish (if we have time). Hopefully, you are starting to realize how easy it really is to interact with remote servers and to parse JSON data. I look forward to seeing you in the next tutorial!

[Previous Lesson](#) [Donate](#) [Next Lesson](#)

Author: trav
I'm just an average guy that love programming.

Share This Post On

Submit a Comment

Your email address will not be published. Required fields are marked *

Name *

Email *

Website

CAPTCHA Image





CAPTCHA Code *

Comment

You may use these HTML tags and attributes: <abbr title=""> <acronym title=""> <blockquote cite=""> <cite> <code> <del datetime=""> <i> <q cite=""> <strike>

Submit Comment